```
'use strict';
/* ============================================================
   赛球弹珠竞速 · Marble Rush — 微信小游戏版
   由 H5 单文件版移植：游戏核心复用，UI 改为 Canvas 绘制。
   ============================================================ */

/* ================ 平台：画布 / 屏幕 / 存储 / 音频 ================ */
const SYS = wx.getSystemInfoSync();
const cw = SYS.windowWidth;
const ch = SYS.windowHeight;
const dpr = Math.min(SYS.pixelRatio || 1, 2);
const canvas = wx.createCanvas();
canvas.width = Math.round(cw * dpr);
canvas.height = Math.round(ch * dpr);
const ctx = canvas.getContext('2d');

const IS_TOUCH = true;
let touchSeen = true;

const store = {
  get(key, def) {
    try {
      const v = wx.getStorageSync(key);
      return (v === '' || v === undefined || v === null) ? def : v;
    } catch (e) { return def; }
  },
  set(key, val) { try { wx.setStorageSync(key, val); } catch (e) { /* 忽略 */ } },
};

let AC = null, master = null, muted = false, lastThock = 0;
function ensureAudio() {
  if (AC) { if (AC.state === 'suspended' && AC.resume) AC.resume(); return; }
  try {
    AC = wx.createWebAudioContext();
    if (AC) {
      master = AC.createGain();
      master.gain.value = settings.vol;
      master.connect(AC.destination);
    }
  } catch (e) { AC = null; }
}
function applyVolume() { if (master) master.gain.value = settings.vol; }

/* ================ 工具 ================ */
const TAU = Math.PI * 2;
const clamp = (v, a, b) => v < a ? a : v > b ? b : v;
const lerp = (a, b, t) => a + (b - a) * t;
function mulberry32(a) { return function () { a |= 0; a = a + 0x6D2B79F5 | 0; let t = Math.imul(a ^ a >>> 15
↪ , 1 | a); t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t; return ((t ^ t >>> 14) >>> 0) / 4294967296; }; }
function fmtT(t) { const m = Math.floor(t / 60), s = t - m * 60; return m + ':' + (s < 10 ? '0' : '') + s.to
```

```
    ↪ Fixed(2); }
0050
0051  /* ================ 音效 ================ */
0052  function tone(f0, f1, dur, type, vol, delay) {
0053    if (!AC || muted) return;
0054    try {
0055      const t = AC.currentTime + (delay || 0);
0056      const o = AC.createOscillator(), g = AC.createGain();
0057      o.type = type || 'sine';
0058      o.frequency.setValueAtTime(Math.max(f0, 1), t);
0059      o.frequency.exponentialRampToValueAtTime(Math.max(f1, 1), t + dur);
0060      g.gain.setValueAtTime(vol || 0.25, t);
0061      g.gain.exponentialRampToValueAtTime(0.001, t + dur);
0062      o.connect(g); g.connect(master);
0063      o.start(t); o.stop(t + dur + 0.03);
0064    } catch (e) { /* 音频失败不影响游戏 */ }
0065  }
0066  function sndImpact(imp) {
0067    const now = Date.now();
0068    if (now - lastThock < 45) return; lastThock = now;
0069    const v = Math.min(0.5, imp / 900);
0070    tone(150 + imp * 0.15, 55, 0.09, 'triangle', v);
0071    tone(900, 300, 0.03, 'square', v * 0.25);
0072  }
0073  const sndBumper = () => tone(220, 700, 0.16, 'square', 0.22);
0074  const sndHole = () => tone(520, 70, 0.5, 'sawtooth', 0.18);
0075  const sndBeep = () => tone(440, 440, 0.12, 'sine', 0.3);
0076  const sndGo = () => tone(660, 990, 0.3, 'square', 0.3);
0077  function sndFinish(win) {
0078    const seq = win ? [523, 659, 784, 1047, 1319] : [523, 659, 784];
0079    seq.forEach((f, i) => tone(f, f, 0.22, 'triangle', 0.3, i * 0.13));
0080  }
0081
0082  /* ================ 设置（本地保存） ================ */
0083  let settings = { ctrl: 'joy', vol: 0.45, mute: false };
0084  let settingsOpen = false;
0085  function loadSettings() {
0086    const s = store.get('marbleSettings', null);
0087    if (s && typeof s === 'object') {
0088      if (s.ctrl === 'btn') settings.ctrl = 'btn';
0089      if (typeof s.vol === 'number') settings.vol = clamp(s.vol, 0, 1);
0090      settings.mute = !!s.mute;
0091    }
0092    muted = settings.mute;
0093  }
0094  function saveSettings() { store.set('marbleSettings', settings); }
0095
0096  /* ================ 常量 ================ */
0097  const W = 880;
0098  const WALL = 26;
```

```
0099  const SEG = 345;
0100  const R = 19;
0101  const GY = 560;
0102  const DRAG = 1.05;
0103  const MAXV = 950;
0104  const GOLD = '#ffd54a', CYAN = '#6cd0ff';
0105  const BALL_POOL = [
0106    { name: '你', color: '#ffd54a' },
0107    { name: '赤焰', color: '#ff5d5d' },
0108    { name: '碧波', color: '#4da3ff' },
0109    { name: '紫电', color: '#b06cff' },
0110    { name: '翠影', color: '#52e0a0' },
0111    { name: '橙旋', color: '#ff9d3d' },
0112    { name: '白霜', color: '#dfe8ff' },
0113    { name: '青岚', color: '#35d0e8' },
0114    { name: '粉黛', color: '#ff7ac8' },
0115    { name: '琥珀', color: '#cd8b45' },
0116  ];
0117  let ballCount = 6;
0118  let customNames = BALL_POOL.map(e => e.name);
0119  function saveRoster() { store.set('marbleRoster', { c: ballCount, n: customNames }); }
0120  function loadRoster() {
0121    const r = store.get('marbleRoster', null);
0122    if (r && Array.isArray(r.n)) {
0123      ballCount = clamp(+r.c || 6, 1, 10);
0124      customNames = BALL_POOL.map((e, i) => (typeof r.n[i] === 'string' && r.n[i].trim()) ? r.n[i].slice(0, 6)
     ↪  : e.name);
0125    }
0126  }
0127  const LENGTHS = {
0128    short: { label: '短道', len: 17000 },
0129    medium: { label: '中道', len: 32000 },
0130    long: { label: '长道', len: 40000 },
0131    xl: { label: '超长', len: 54000 },
0132  };
0133  const AILVLS = {
0134    easy: { label: '简单', scale: 0.35, spd: 0.88, ai: [0.50, 0.57, 0.64, 0.70, 0.76] },
0135    normal: { label: '普通', scale: 0.65, spd: 0.97, ai: [0.70, 0.76, 0.82, 0.87, 0.91] },
0136    hard: { label: '困难', scale: 0.85, spd: 1.12, ai: [0.90, 0.93, 0.95, 0.965, 0.98] },
0137    nightmare: { label: '噩梦', scale: 1.00, spd: 1.20, ai: [0.97, 0.98, 0.985, 0.99, 0.995] },
0138  };
0139
0140  /* ================= 全局状态 ================= */
0141  let state = 'menu';
0142  let paused = false;
0143  let mode = '1p';
0144  let simTime = 0, raceTime = 0, countdownT = 0, goT = -99, lastCount = 99;
0145  let seed = (Math.random() * 1e9) | 0, lenKey = 'medium', diffKey = 'normal';
0146  let trackLen = LENGTHS.medium.len;
0147  let track = null, balls = [], players = [];
```

```
let menuCam = 0;
let particles = [];
let finishWaitEnd = 0, resultsFinal = false, lastResUpdate = 0;
let clickTargets = [];
let spectateSplit = false;
let backPrompt = false;
let editingName = -1;

/* ================ 赛道生成 ================ */
function pat_gate(y, d, out, rng) {
  const gap = lerp(200, 136, d) + rng() * 22;
  const gx = lerp(WALL + gap / 2 + 40, W - WALL - gap / 2 - 40, rng());
  const th = 26;
  const wl = (gx - gap / 2) - WALL, wr = (W - WALL) - (gx + gap / 2);
  if (wl > 8) out.push({ type: 'rect', x: WALL, y, w: wl, h: th });
  if (wr > 8) out.push({ type: 'rect', x: gx + gap / 2, y, w: wr, h: th });
  return th;
}
function pat_slalom(y, d, out, rng) {
  const n = 3 + Math.round(d * 1.5);
  for (let i = 0; i < n; i++) {
    const left = i % 2 === 0;
    out.push({ type: 'bumper', cx: left ? lerp(W * 0.28, W * 0.42, rng()) : lerp(W * 0.58, W * 0.72, rng()),
↪  cy: y + 45 + i * 130, r: 28 });
  }
  return 45 + (n - 1) * 130 + 30;
}
function pat_pinball(y, d, out, rng) {
  const rows = 3 + (d > 0.45 ? 1 : 0) + (d > 0.8 ? 1 : 0);
  const x0 = WALL + 80, step = (W - 2 * WALL - 160) / 5;
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < 6; c++) {
      if (rng() < 0.25) continue;
      out.push({ type: 'peg', cx: x0 + c * step + (r % 2 ? step / 2 : 0), cy: y + 45 + r * 112, r: 11 });
    }
  return 45 + (rows - 1) * 112 + 30;
}
function pat_windmill(y, d, out, rng) {
  const len = lerp(210, 340, d);
  const spd = (1.2 + 1.6 * d) * (rng() < 0.5 ? -1 : 1);
  const two = d > 0.30 && rng() < 0.6;
  const len2 = two ? lerp(190, 290, d) : 0;
  out.push({ type: 'rot', mode: 'spin', cx: lerp(W * 0.35, W * 0.65, rng()), cy: y + 75, len, thick: 18, spe
↪ ed: spd, phase: rng() * TAU });
  if (two) out.push({ type: 'rot', mode: 'spin', cx: lerp(W * 0.30, W * 0.40, rng()), cy: y + 75 + 90, len:
↪ len2, thick: 18, speed: -spd, phase: rng() * TAU });
  return 75 + (two ? 90 + len2 / 2 : len / 2) + 40;
}
function pat_pistons(y, d, out, rng) {
  const mk = (side, yy, amp, speed, phase) => ({ type: 'piston', side, y: yy, h: 34, amp, min: 36, speed, ph
```

```
    ↪ ase });
0195    if (rng() < 0.35) {
0196      const amp = 130 + 60 * d;
0197      out.push({ type: 'rect', x: W / 2 - 70, y: y + 20, w: 140, h: 150 });
0198      out.push(mk('L', y + 55, amp, 1.7, 0));
0199      out.push(mk('R', y + 55, amp, 1.7, Math.PI));
0200      return 210;
0201    } else {
0202      let side = rng() < 0.5 ? 'L' : 'R';
0203      const n = 2 + (d > 0.40 ? 1 : 0) + (d > 0.75 ? 1 : 0);
0204      for (let i = 0; i < n; i++) {
0205        out.push(mk(side, y + i * 150, 130 + 90 * d, 1.3 + 0.8 * d, rng() * TAU));
0206        side = side === 'L' ? 'R' : 'L';
0207      }
0208      return (n - 1) * 150 + 70;
0209    }
0210  }
0211  function pat_holes(y, d, out, rng) {
0212    const n = 2 + Math.round(d * 2);
0213    for (let i = 0; i < n; i++)
0214      out.push({ type: 'hole', cx: lerp(WALL + 105, W - WALL - 105, rng()), cy: y + 50 + i * 100 + (rng() - 0.
    ↪ 5) * 36, r: 26 + 16 * d });
0215    return 50 + (n - 1) * 100 + 60;
0216  }
0217  function pat_funnel(y, d, out, rng) {
0218    const g = lerp(215, 175, d) + rng() * 18;
0219    const c = W / 2 + (rng() - 0.5) * 150;
0220    const L = 330;
0221    out.push({ type: 'rot', mode: 'static', cx: c - g / 2 - L * 0.42, cy: y + 85, len: L, thick: 20, angle: 0.
    ↪ 55 });
0222    out.push({ type: 'rot', mode: 'static', cx: c + g / 2 + L * 0.42, cy: y + 85, len: L, thick: 20, angle: -0
    ↪ .55 });
0223    return 290;
0224  }
0225  function pat_hammer(y, d, out, rng) {
0226    const len = 260 + 90 * d, speed = 1.5 + 0.9 * d;
0227    out.push({ type: 'rot', mode: 'swing', cx: WALL + 6, cy: y + 55, len, thick: 18, center: 0.35, amp: 0.85,
    ↪ speed, phase: rng() * TAU });
0228    if (d > 0.50 && rng() < 0.55)
0229      out.push({ type: 'rot', mode: 'swing', cx: W - WALL - 6, cy: y + 55, len, thick: 18, center: Math.PI - 0
    ↪ .35, amp: 0.85, speed, phase: rng() * TAU });
0230    return 55 + len / 2 + 40;
0231  }
0232  function pat_double(y, d, out, rng) {
0233    pat_gate(y, Math.min(1, d + 0.1), out, rng);
0234    pat_windmill(y + 190, d, out, rng);
0235    return 190 + 75 + lerp(210, 340, d) / 2 + 40;
0236  }
0237  function pat_zigzag(y, d, out, rng) {
0238    const n = 2 + (d > 0.65 ? 1 : 0);
```

```
  let side = rng() < 0.5;
  for (let i = 0; i < n; i++) {
    const gap = lerp(215, 158, d) + rng() * 16;
    const gx = side ? lerp(WALL + gap / 2 + 38, W * 0.30, rng())
                    : lerp(W * 0.70, W - WALL - gap / 2 - 38, rng());
    const wl = (gx - gap / 2) - WALL, wr = (W - WALL) - (gx + gap / 2);
    if (wl > 8) out.push({ type: 'rect', x: WALL, y: y + i * 230, w: wl, h: 26 });
    if (wr > 8) out.push({ type: 'rect', x: gx + gap / 2, y: y + i * 230, w: wr, h: 26 });
    side = !side;
  }
  return (n - 1) * 230 + 60;
}
const PATS = { gate: pat_gate, slalom: pat_slalom, pinball: pat_pinball, windmill: pat_windmill,
  pistons: pat_pistons, holes: pat_holes, funnel: pat_funnel, hammer: pat_hammer,
  double: pat_double, zigzag: pat_zigzag };

function genTrack(sd, scaleD, len) {
  const rng = mulberry32(sd);
  const out = [];
  const pick = arr => { let s = 0; for (const a of arr) s += a[1]; let r = rng() * s; for (const a of arr) {
↪  r -= a[1]; if (r <= 0) return a[0]; } return arr[0][0]; };
  let y = 560;
  const d = Math.min(scaleD, 1);
  let lastKind = '';
  while (y < len - 650) {
    let kind = pick([['gate', 0.9], ['slalom', 0.9], ['pinball', 0.9], ['windmill', 1], ['funnel', 0.65],
      ['pistons', d > 0.06 ? 1 : 0.20], ['holes', d > 0.12 ? 0.9 : 0.15], ['hammer', d > 0.22 ? 1 : 0.08],
      ['double', d > 0.4 ? 0.8 : 0], ['zigzag', 0.55]]);
    if (kind === lastKind)
      kind = pick([['gate', 0.9], ['slalom', 0.9], ['pinball', 0.9], ['windmill', 1], ['funnel', 0.65],
        ['pistons', d > 0.06 ? 1 : 0.20], ['holes', d > 0.12 ? 0.9 : 0.15], ['hammer', d > 0.22 ? 1 : 0.08],
        ['double', d > 0.4 ? 0.8 : 0], ['zigzag', 0.55]]);
    lastKind = kind;
    const h = PATS[kind](y, d, out, rng) || 60;
    y += Math.max(330, h + 150 + rng() * 60);
  }
  out.sort((a, b) => ((a.y ?? a.cy) - (b.y ?? b.cy)));
  for (const o of out) {
    if (o.type === 'rect') { o._cy = o.y + o.h / 2; o._r = Math.hypot(o.w, o.h) / 2 + 12; }
    else if (o.type === 'piston') { o._cy = o.y + o.h / 2; o._r = o.min + o.amp + o.h; }
    else if (o.type === 'rot') { o._cy = o.cy; o._r = o.len / 2 + 22; }
    else { o._cy = o.cy; o._r = o.r + 12; }
  }
  out.sort((a, b) => a._cy - b._cy);
  return { obstacles: out };
}

/* ================= 机关运动学 ================= */
function angleOmega(o, t) {
  if (o.mode === 'spin') return [o.phase + o.speed * t, o.speed];
```

```
    if (o.mode === 'swing') { const ph = t * o.speed + o.phase; return [o.center + o.amp * Math.sin(ph), o.amp * o.speed * Math.cos(ph)]; }
    return [o.angle, 0];
  }
  function ensureKin(o) {
    if (o._kt === simTime) return;
    o._kt = simTime;
    if (o.type === 'rot') { const r = angleOmega(o, simTime); o._a = r[0]; o._w = r[1]; }
    else if (o.type === 'piston') {
      const ph = simTime * o.speed + o.phase, k = 0.5 + 0.5 * Math.sin(ph);
      const reach = o.min + o.amp * k;
      o._vx = o.amp * 0.5 * o.speed * Math.cos(ph) * (o.side === 'L' ? 1 : -1);
      o._rect = o.side === 'L' ? { x: WALL, y: o.y, w: reach, h: o.h } : { x: W - WALL - reach, y: o.y, w: reach, h: o.h };
    }
  }
  function obsWindow(y0, y1) {
    const obs = track.obstacles;
    const lo0 = y0 - 360, hi0 = y1 + 360;
    let lo = 0, hi = obs.length;
    while (lo < hi) { const m = (lo + hi) >> 1; if (obs[m]._cy < lo0) lo = m + 1; else hi = m; }
    const i0 = lo;
    hi = obs.length;
    while (lo < hi) { const m = (lo + hi) >> 1; if (obs[m]._cy <= hi0) lo = m + 1; else hi = m; }
    return [i0, lo];
  }
  function rotNearest(o, px, py) {
    const c = Math.cos(o._a), s = Math.sin(o._a);
    const dx = px - o.cx, dy = py - o.cy;
    const lx = dx * c + dy * s, ly = -dx * s + dy * c;
    const qx = clamp(lx, -o.len / 2, o.len / 2), qy = clamp(ly, -o.thick / 2, o.thick / 2);
    return [o.cx + qx * c - qy * s, o.cy + qx * s + qy * c, lx, ly];
  }

  /* ================= 弹珠 ================= */
  function makeBalls() {
    const jr = mulberry32(seed ^ 0x5f3759df);
    const n = mode === '2p' ? Math.max(2, ballCount) : Math.max(1, ballCount);
    const roster = BALL_POOL.map((e, i) => ({ name: customNames[i].trim() || e.name, color: e.color }));
    players = [];
    balls = [];
    if (mode === 'spectate') {
      for (let i = 0; i < n; i++)
        balls.push(mkBall(false, -1, roster[i].name, roster[i].color, 0.93 + jr() * 0.06, i, n, jr));
      players = [balls[0]];
      return balls;
    }
    const arr = AILVLS[diffKey].ai, lo = arr[0], hi = arr[arr.length - 1];
    const pSlots = mode === '2p'
      ? [Math.round((n - 1) * 0.3), Math.round((n - 1) * 0.7)]
```

```
    : [Math.floor((n - 1) / 2)];
  const aiDefs = roster.slice(pSlots.length);
  let ai = 0;
  for (let i = 0; i < n; i++) {
    const ps = pSlots.indexOf(i);
    if (ps >= 0) {
      balls.push(mkBall(true, ps, roster[ps].name, roster[ps].color, 1, i, n, jr));
      players.push(balls[balls.length - 1]);
    } else {
      const d = aiDefs[ai % aiDefs.length];
      const t = n - pSlots.length > 1 ? ai / (n - pSlots.length - 1) : 0;
      const skill = clamp(hi - (hi - lo) * t + (jr() - 0.5) * 0.04, 0.40, 0.99);
      balls.push(mkBall(false, -1, d.name, d.color, skill, i, n, jr));
      ai++;
    }
  }
  return balls;
}
function mkBall(isP, pi, name, color, skill, slot, total, jr) {
  return { isPlayer: isP, pi, name, color, skill,
    x: lerp(WALL + 95, W - WALL - 95, total === 1 ? 0.5 : slot / (total - 1)), y: 118, vx: 0, vy: 0,
    phase: jr() * TAU, wf: 0.5 + jr() * 1.1, wa: (1 - skill) * 60 + 8, laneOff: (jr() - 0.5) * 76,
    finished: false, finishTime: 0, falls: 0, slowT: 0, stuck2: 0, falling: null, fallScale: 1,
    trail: [], spin: 0, _camY: null, _shake: 0 };
}

/* ================ 碰撞 ================ */
function resolveCircleRect(b, rect, vpx, vpy) {
  const cx = clamp(b.x, rect.x, rect.x + rect.w), cy = clamp(b.y, rect.y, rect.y + rect.h);
  let dx = b.x - cx, dy = b.y - cy, d2 = dx * dx + dy * dy;
  if (d2 >= R * R) return;
  let nx, ny, pen;
  if (d2 > 1e-6) { const d = Math.sqrt(d2); nx = dx / d; ny = dy / d; pen = R - d; }
  else {
    const dl = b.x - rect.x, dr = rect.x + rect.w - b.x, dt2 = b.y - rect.y, db = rect.y + rect.h - b.y;
    const m = Math.min(dl, dr, dt2, db);
    nx = m === dl ? -1 : m === dr ? 1 : 0; ny = m === dt2 ? -1 : m === db ? 1 : 0; pen = R + m;
  }
  b.x += nx * pen; b.y += ny * pen;
  const rvx = b.vx - (vpx || 0), rvy = b.vy - (vpy || 0);
  const vn = rvx * nx + rvy * ny;
  if (vn < 0) {
    const e = 0.22;
    b.vx = rvx - (1 + e) * vn * nx + (vpx || 0);
    b.vy = rvy - (1 + e) * vn * ny + (vpy || 0);
    if (-vn > 150) onImpact(b, -vn);
  }
}
function resolveRot(b, o) {
  const [wx, wy, lx, ly] = rotNearest(o, b.x, b.y);
```

```
    const dx = b.x - wx, dy = b.y - wy, d2 = dx * dx + dy * dy;
    if (d2 >= R * R) return;
    let nx, ny;
    if (d2 > 1e-6) { const d = Math.sqrt(d2); nx = dx / d; ny = dy / d; }
    else { const sg = ly >= 0 ? 1 : -1; const s = Math.sin(o._a), c = Math.cos(o._a); nx = -sg * s; ny = sg *
↪ c; }
    b.x += nx * R; b.y += ny * R;
    const rx = wx - o.cx, ry = wy - o.cy;
    const svx = o._w * -ry, svy = o._w * rx;
    const rvx = b.vx - svx, rvy = b.vy - svy;
    const vn = rvx * nx + rvy * ny;
    if (vn < 0) {
      const e = 0.30;
      b.vx = rvx - (1 + e) * vn * nx + svx;
      b.vy = rvy - (1 + e) * vn * ny + svy;
      if (-vn > 150) onImpact(b, -vn);
    }
  }
  function resolveCircleObst(b, o) {
    const dx = b.x - o.cx, dy = b.y - o.cy, rr = R + o.r;
    const d2 = dx * dx + dy * dy;
    if (d2 >= rr * rr) return;
    const d = Math.sqrt(d2) || 0.001, nx = dx / d, ny = dy / d;
    b.x += nx * (rr - d); b.y += ny * (rr - d);
    const vn = b.vx * nx + b.vy * ny;
    if (vn < 0) {
      const e = o.type === 'bumper' ? 0.75 : 0.4;
      b.vx -= (1 + e) * vn * nx; b.vy -= (1 + e) * vn * ny;
      if (o.type === 'bumper') {
        b.vx += nx * 200; b.vy += ny * 200;
        o.hitT = simTime; sndBumper();
        spawnSparks(b.x, b.y, 6, '#ff9d9d');
      } else if (-vn > 200) onImpact(b, -vn);
    }
  }
  function onImpact(b, imp) {
    if (imp > 420) spawnSparks(b.x, b.y, Math.min(10, imp / 90 | 0), '#ffe9a8');
    if (b.isPlayer) {
      if (imp > 160) sndImpact(imp);
      if (imp > 560) b._shake = Math.min(13, imp / 90);
    } else if (imp > 300) sndImpact(imp * 0.35);
  }
  function spawnSparks(x, y, n, color) {
    for (let i = 0; i < n; i++) {
      const a = Math.random() * TAU, sp = 90 + Math.random() * 260;
      particles.push({ x, y, vx: Math.cos(a) * sp, vy: Math.sin(a) * sp, life: 0.45, ttl: 0.45, r: 2.2, color
↪ });
    }
  }
  function spawnConfetti(x, y) {
```

```
    for (let i = 0; i < 130; i++) {
      const a = Math.random() * TAU, sp = 120 + Math.random() * 420;
      particles.push({ x, y, vx: Math.cos(a) * sp, vy: Math.sin(a) * sp - 140, life: 1.6, ttl: 1.6,
        r: 3 + Math.random() * 3, color: `hsl(${Math.random() * 360},90%,65%)` });
    }
  }
  function spotFree(x, y) {
    if (x < WALL + R + 2 || x > W - WALL - R - 2) return false;
    const [w0, w1] = obsWindow(y - 60, y + 60);
    for (let k = w0; k < w1; k++) {
      const o = track.obstacles[k];
      if (Math.abs(o._cy - y) > o._r + R + 10) continue;
      if (o.type === 'rect' || o.type === 'piston') {
        ensureKin(o);
        const r = o._rect || o;
        if (x > r.x - R - 2 && x < r.x + r.w + R + 2 && y > r.y - R - 2 && y < r.y + r.h + R + 2) return false
    ↪ ;
      } else if (o.type === 'rot') {
        ensureKin(o);
        const [wx, wy] = rotNearest(o, x, y);
        if ((x - wx) ** 2 + (y - wy) ** 2 < (R + 4) ** 2) return false;
      } else if (o.type === 'peg' || o.type === 'bumper') {
        if ((x - o.cx) ** 2 + (y - o.cy) ** 2 < (R + o.r + 5) ** 2) return false;
      } else if (o.type === 'hole') {
        if ((x - o.cx) ** 2 + (y - o.cy) ** 2 < (o.r + R + 9) ** 2) return false;
      }
    }
    return true;
  }

  /* ================ AI ================ */
  function obstacleSpan(o) {
    if (o.type === 'rect') return [o.y, o.y + o.h, o.x, o.x + o.w];
    if (o.type === 'piston') {
      const ext = o.min + o.amp * 0.55;
      return [o.y, o.y + o.h, o.side === 'L' ? WALL : W - WALL - ext, o.side === 'L' ? WALL + ext : W - WALL];
    }
    if (o.type === 'rot') {
      const hl = o.len / 2;
      return [o.cy - hl, o.cy + hl, o.cx - hl, o.cx + hl];
    }
    const r = o.r;
    return [o.cy - r, o.cy + r, o.cx - r, o.cx + r];
  }
  function collectRows(b, look) {
    const ahead = [];
    const [w0, w1] = obsWindow(b.y - 400, b.y + look);
    for (let k = w0; k < w1; k++) {
      const o = track.obstacles[k];
      if (o._cy - o._r > b.y + look + 60 || o._cy + o._r < b.y - 30) continue;
```

```
    const s = obstacleSpan(o);
    if (s[1] < b.y - 30 || s[0] > b.y + look) continue;
    ahead.push(s);
  }
  if (!ahead.length) return [];
  ahead.sort((p, q) => p[0] - q[0]);
  const rows = [];
  let i = 0;
  while (i < ahead.length && rows.length < 3) {
    const rowY = ahead[i][0];
    let j = i;
    while (j < ahead.length && ahead[j][0] < rowY + 60) j++;
    rows.push({ y: rowY, spans: ahead.slice(i, j) });
    i = j;
  }
  return rows;
}
function rowIntervals(row, L, Rt) {
  const spans = [];
  for (const s of row.spans) {
    const a = Math.max(s[2] - 12, L), c2 = Math.min(s[3] + 12, Rt);
    if (c2 > a) spans.push([a, c2]);
  }
  spans.sort((p, q) => p[0] - q[0]);
  const ivs = []; let cur = L;
  for (const [s1, s2] of spans) {
    if (s1 > cur) ivs.push([cur, s1]);
    cur = Math.max(cur, s2);
  }
  if (cur < Rt) ivs.push([cur, Rt]);
  return ivs;
}
function aiForce(b) {
  const P = AILVLS[diffKey];
  let fx = Math.sin(simTime * b.wf + b.phase) * b.wa * 0.5;
  let fy = GY * (P.spd - 0.06 + 0.10 * b.skill);
  const look = 280 + 560 * b.skill;
  const L = WALL + R + 4, Rt = W - WALL - R - 4;
  const rows = collectRows(b, look);
  if (rows.length) {
    for (const r of rows) {
      r.ivs = rowIntervals(r, L, Rt);
      r.pass = r.ivs.filter(iv => iv[1] - iv[0] >= 2 * R + 8);
    }
    const depth = Math.min(rows.length, 1 + Math.round(b.skill * 2.2));
    const use = rows.slice(0, depth);
    let target = null, bestScore = 1e18;
    for (const iv of use[0].pass) {
      const c = (iv[0] + iv[1]) / 2;
      let score = Math.abs(c - b.x) + Math.abs(c - W / 2) * 0.2;
```

```
      let cur = c, dead = false;
      for (let ri = 1; ri < use.length; ri++) {
        const vGap = Math.max(40, use[ri].y - use[ri - 1].y);
        let bn = null;
        for (const jv of use[ri].pass) {
          const c2 = (jv[0] + jv[1]) / 2, dx = Math.abs(c2 - cur);
          if (dx < vGap * 1.15 + 80 && (!bn || dx < bn.dx)) bn = { c: c2, dx };
        }
        if (!bn) { dead = true; break; }
        score += bn.dx * 0.7; cur = bn.c;
      }
      if (dead) score += 2500;
      if (score < bestScore) { bestScore = score; target = c; }
    }
    if (target === null) {
      let wmax = 0;
      for (const iv of use[0].ivs) { const w = iv[1] - iv[0]; if (w > wmax) { wmax = w; target = (iv[0] + iv[1]) / 2; } }
      if (target === null) target = W / 2;
    }
    target = clamp(target + b.laneOff * (0.6 - 0.45 * b.skill), L + 8, Rt - 8);
    const steer = 430 + 520 * b.skill;
    fx += clamp((target - b.x) * 9 - b.vx * 1.4, -steer, steer);
    const rowDist = use[0].y - b.y;
    if (rowDist >= 0 && rowDist < 200 && Math.abs(target - b.x) > 140)
      fy *= lerp(0.55, 0.88, b.skill);
  }
  for (const o of balls) {
    if (o === b || o.falling !== null) continue;
    const dx = o.x - b.x, dy = o.y - b.y;
    if (dy > -10 && dy < 130 && Math.abs(dx) < 56)
      fx += (dx >= 0 ? -1 : 1) * 260 * (1 - dy / 130);
  }
  if (!rows.length) fx += clamp((W / 2 - b.x) * 1.2, -180, 180);
  const m = 80;
  if (b.x < WALL + m) fx += (1 - (b.x - WALL) / m) * 520;
  if (b.x > W - WALL - m) fx -= (1 - (W - WALL - b.x) / m) * 520;
  return [clamp(fx, -1700, 1700), fy];
}

/* ================ 输入状态 ================ */
const keys1 = { left: 0, right: 0, up: 0, down: 0 };
const keys2 = { left: 0, right: 0, up: 0, down: 0 };
const joys = [{ pid: null, bx: 0, by: 0, dx: 0, dy: 0, x: 0, y: 0 }, { pid: null, bx: 0, by: 0, dx: 0, dy: 0, x: 0, y: 0 }];
const JOY_R = 58;
const btnState = [{ left: 0, right: 0 }, { left: 0, right: 0 }];
const btnPresses = new Map();
function syncBtnState() {
  btnState[0].left = btnState[0].right = 0;
```

```
  btnState[1].left = btnState[1].right = 0;
  for (const pr of btnPresses.values()) btnState[pr.p][pr.key] = 1;
}
function effKeys(b) {
  if (mode === '2p') return b.pi === 0 ? keys1 : keys2;
  return { left: keys1.left | keys2.left, right: keys1.right | keys2.right,
    up: keys1.up | keys2.up, down: keys1.down | keys2.down };
}

/* ================ 物理 ================ */
function physics(dt) {
  for (const b of balls) {
    if (b.falling !== null) {
      b.falling -= dt; b.fallScale = Math.max(0.05, b.falling / 0.7);
      if (b.falling <= 0) {
        const h = b.fallHole;
        let ry = h.cy + h.r + R + 10;
        for (let i = 0; i < 24; i++) { if (spotFree(b.x, ry)) break; ry += 12; }
        b.x = clamp(b.x, WALL + R + 6, W - WALL - R - 6); b.y = ry;
        b.vx = 0; b.vy = 70; b.falling = null; b.fallScale = 1;
      }
      continue;
    }
    let fx = 0, fy = 0;
    if (!b.finished) {
      if (b.isPlayer) {
        const k = effKeys(b);
        const pi = mode === '2p' && b.pi === 1 ? 1 : 0;
        const j = joys[pi];
        let sx = (k.right - k.left);
        let brake = k.up;
        if (settings.ctrl === 'btn') {
          const bs = btnState[pi];
          if (bs.right > bs.left) sx = 1; else if (bs.left > bs.right) sx = -1;
        } else if (j.pid !== null) {
          if (Math.abs(j.x) > 0.12) sx = j.x;
          if (j.y < -0.45) brake = true;
        }
        fx = sx * 820;
        fy = GY - (brake ? 520 : 0);
      } else { const f = aiForce(b); fx = f[0]; fy = f[1]; }
    } else { fy = GY * 0.25; }
    b.vx += fx * dt; b.vy += fy * dt;
    const drag = b.finished ? 3 : DRAG, f = Math.exp(-drag * dt);
    b.vx *= f; b.vy *= f;
    const sp = Math.hypot(b.vx, b.vy);
    if (sp > MAXV) { b.vx *= MAXV / sp; b.vy *= MAXV / sp; }
    b.x += b.vx * dt; b.y += b.vy * dt;
    b.spin += (sp / R) * dt * (b.vx >= 0 ? 1 : -1) * 0.6;
    if (!b.finished && b.y < trackLen) {
```

```
      b._progT = (b._progT || 0) + dt;
      if (b._progT >= 0.8) {
        const gain = b.y - (b._lastY ?? b.y);
        if (gain < 30) {
          b.stuck2++;
          if (b.stuck2 >= 2) {
            let found = null;
            search:
            for (const dist of [70, 140, 220, 320]) {
              for (let k = 0; k < 14; k++) {
                const ang = Math.random() * TAU;
                const nx = clamp(b.x + Math.cos(ang) * dist, WALL + R + 4, W - WALL - R - 4);
                const ny = b.y + Math.sin(ang) * dist;
                if (ny > b.y - 40 && spotFree(nx, ny)) { found = [nx, ny]; break search; }
              }
            }
            if (found) { b.x = found[0]; b.y = found[1]; b.vx = 0; b.vy = 140; b.trail.length = 0; }
            else { b.y += 160; b.vy = 140; b.vx *= 0.3; }
            b.stuck2 = 0;
          } else { b.vy += 320; b.vx += (Math.random() - 0.5) * 300; }
        } else b.stuck2 = Math.max(0, b.stuck2 - 1);
        b._lastY = b.y; b._progT = 0;
      }
      if (Math.abs(b.vy) < 90) {
        b.slowT += dt;
        if (b.slowT > 0.8) { b.vy += 420; b.vx += (Math.random() - 0.5) * 260; b.slowT = 0.25; }
      } else b.slowT = 0;
    } else { b.slowT = 0; b.stuck2 = 0; }
  }
  for (let it = 0; it < 2; it++) {
    for (const b of balls) {
      if (b.falling !== null) continue;
      const [w0, w1] = obsWindow(b.y - 660, b.y + 660);
      for (let k = w0; k < w1; k++) {
        const o = track.obstacles[k];
        if (Math.abs(o._cy - b.y) > o._r + 320) continue;
        if (o.type === 'rect') resolveCircleRect(b, o, 0, 0);
        else if (o.type === 'piston') { ensureKin(o); resolveCircleRect(b, o._rect, o._vx, 0); }
        else if (o.type === 'rot') { ensureKin(o); resolveRot(b, o); }
        else resolveCircleObst(b, o);
      }
      if (b.x < WALL + R) { b.x = WALL + R; if (b.vx < 0) b.vx *= -0.5; }
      if (b.x > W - WALL - R) { b.x = W - WALL - R; if (b.vx > 0) b.vx *= -0.5; }
    }
    for (let i = 0; i < balls.length; i++) for (let j = i + 1; j < balls.length; j++) {
      const a = balls[i], c = balls[j];
      if (a.falling !== null || c.falling !== null) continue;
      const dx = c.x - a.x, dy = c.y - a.y, d2 = dx * dx + dy * dy, rr = 2 * R;
      if (d2 >= rr * rr || d2 === 0) continue;
      const d = Math.sqrt(d2), nx = dx / d, ny = dy / d, pen = (rr - d) / 2;
```

```
      a.x -= nx * pen; a.y -= ny * pen; c.x += nx * pen; c.y += ny * pen;
      const rel = (a.vx - c.vx) * nx + (a.vy - c.vy) * ny;
      if (rel > 0) {
        const imp = (1 + 0.55) * rel / 2;
        a.vx -= imp * nx; a.vy -= imp * ny; c.vx += imp * nx; c.vy += imp * ny;
        if (Math.abs(ny) > 0.75 && rel < 140) {
          const kick = (Math.random() < 0.5 ? 1 : -1) * 130;
          a.vx += kick; c.vx -= kick;
        }
        if (rel > 240 && (a.isPlayer || c.isPlayer)) { sndImpact(rel); spawnSparks((a.x + c.x) / 2, (a.y + c.y) / 2, 4, '#fff'); }
      }
    }
  }
  for (const b of balls) {
    if (b.falling !== null || b.finished) continue;
    const [w0, w1] = obsWindow(b.y - 160, b.y + 160);
    for (let k = w0; k < w1; k++) {
      const o = track.obstacles[k];
      if (o.type !== 'hole') continue;
      if (Math.abs(o._cy - b.y) > o._r + R) continue;
      const dx = b.x - o.cx, dy = b.y - o.cy;
      if (dx * dx + dy * dy < (o.r - 4) ** 2) {
        b.falling = 0.7; b.fallHole = o; b.falls++;
        b.vx = 0; b.vy = 0;
        if (b.isPlayer) { sndHole(); b._shake = Math.max(b._shake, 6); }
        break;
      }
    }
  }
  for (const b of balls) {
    if (!b.finished && b.y >= trackLen) {
      b.finished = true; b.finishTime = raceTime;
      if (b.isPlayer) playerFinished(b);
      else if (mode === 'spectate' && state === 'race') {
        state = 'finishing';
        finishWaitEnd = raceTime + 30;
      }
    }
    if (b.y > trackLen + 520) { b.y = trackLen + 520; b.vy = Math.min(b.vy, 0); }
  }
  for (const b of balls) {
    if (b.falling !== null) { b.trail.length = 0; continue; }
    b.trail.push(b.x, b.y);
    if (b.trail.length > 34) b.trail.splice(0, 2);
  }
}

/* ================ 名次 ================ */
function standings() {
```

```
  return balls.slice().sort((a, b) => {
    if (a.finished && b.finished) return a.finishTime - b.finishTime;
    if (a.finished) return -1;
    if (b.finished) return 1;
    return b.y - a.y;
  });
}

/* ================ 流程控制 ================ */
function bestKey() { return 'marbleBest_' + lenKey + '_' + diffKey; }
function bestText() {
  const v = store.get(bestKey(), 0);
  return '最佳 (' + LENGTHS[lenKey].label + '·' + AILVLS[diffKey].label + ') : ' + (v ? fmtT(+v) : '--:--.-');
}
let resultsVisible = false;
let resTitleText = '完赛! ';
let resNoteText = '';

function startRace(newSeed) {
  ensureAudio();
  if (newSeed) seed = (Math.random() * 1e9) | 0;
  trackLen = LENGTHS[lenKey].len;
  track = genTrack(seed, AILVLS[diffKey].scale, trackLen);
  balls = makeBalls();
  raceTime = 0; countdownT = 3.0; lastCount = 99; goT = -99;
  state = 'countdown'; paused = false; resultsFinal = false;
  settingsOpen = false; resultsVisible = false; backPrompt = false;
  btnPresses.clear(); syncBtnState();
  for (const j of joys) { j.pid = null; j.dx = j.dy = j.x = j.y = 0; }
  particles.length = 0;
}
function goMenu() {
  state = 'menu'; paused = false; backPrompt = false;
  settingsOpen = false; resultsVisible = false;
  btnPresses.clear(); syncBtnState();
  particles.length = 0;
}
function playerFinished(b) {
  const st = standings(); const rank = st.indexOf(b) + 1;
  sndFinish(rank === 1);
  if (rank === 1) spawnConfetti(b.x, b.y);
  if (state !== 'finishing') {
    state = 'finishing';
    finishWaitEnd = raceTime + 8;
    resultsVisible = true;
    resTitleText = '完赛! ';
    resNoteText = '其余弹珠仍在冲刺，名次实时更新……';
    for (const j of joys) { j.pid = null; j.dx = j.dy = j.x = j.y = 0; }
    btnPresses.clear(); syncBtnState();
  }
```

```
}
function renderResults(final) {
  const st = standings();
  if (final) {
    resultsFinal = true; state = 'results';
    const winner = st[0];
    if (mode === 'spectate') {
      resTitleText = '🏆 ' + winner.name + ' 夺冠! ';
      resNoteText = '再来一局重新观赛，或换一条新赛道。';
    } else if (mode === '2p') {
      resTitleText = winner.isPlayer ? '🏆 ' + winner.name + ' 冠军! ' : '完赛! ';
      resNoteText = players.map(p => p.name + ' 第 ' + (st.indexOf(p) + 1) + ' 名').join(' · ');
      if (winner.isPlayer) resNoteText += ' — 你们击败了所有 人机! ';
    } else {
      resTitleText = winner.isPlayer ? '🏆 冠军! ' : '完赛! ';
      const r = st.indexOf(players[0]) + 1;
      resNoteText = r === 1 ? '你击败了所有对手! ' : '你获得了第 ' + r + ' 名，再试一次冲击冠军吧! ';
      const prev = store.get(bestKey(), 0);
      const p = players[0];
      if (p.finished && (!prev || p.finishTime < +prev)) store.set(bestKey(), p.finishTime);
    }
    resultsVisible = true;
  }
}

/* ================ 绘制基础 ================ */
function pathRoundRect(x, y, w, h, r) {
  r = Math.min(r, w / 2, h / 2);
  ctx.beginPath();
  ctx.moveTo(x + r, y);
  ctx.lineTo(x + w - r, y); ctx.arcTo(x + w, y, x + w, y + r, r);
  ctx.lineTo(x + w, y + h - r); ctx.arcTo(x + w, y + h, x + w - r, y + h, r);
  ctx.lineTo(x + r, y + h); ctx.arcTo(x, y + h, x, y + h - r, r);
  ctx.lineTo(x, y + r); ctx.arcTo(x, y, x + r, y, r);
  ctx.closePath();
}
function fillRoundRect(x, y, w, h, r, fill, stroke, lw) {
  pathRoundRect(x, y, w, h, r);
  if (fill) { ctx.fillStyle = fill; ctx.fill(); }
  if (stroke) { ctx.strokeStyle = stroke; ctx.lineWidth = lw || 1; ctx.stroke(); }
}
function textCentered(text, x, y, font, color) {
  ctx.font = font; ctx.fillStyle = color; ctx.textAlign = 'center'; ctx.textBaseline = 'alphabetic';
  ctx.fillText(text, x, y);
}
function wrapText(text, x, y, maxW, lineH, font, color) {
  ctx.font = font; ctx.fillStyle = color; ctx.textAlign = 'left'; ctx.textBaseline = 'alphabetic';
  let line = '', yy = y;
  for (const ch2 of text) {
    if (ctx.measureText(line + ch2).width > maxW && line) {
```

```
0830        ctx.fillText(line, x, yy); line = ch2; yy += lineH;
0831      } else line += ch2;
0832    }
0833    if (line) ctx.fillText(line, x, yy);
0834    return yy;
0835  }
0836
0837  /* ================ 世界渲染 ================ */
0838  function drawWorld(yTop, yBot) {
0839    ctx.fillStyle = '#0d1626'; ctx.fillRect(0, yTop, W, yBot - yTop);
0840    ctx.fillStyle = 'rgba(140,190,255,0.028)';
0841    for (let gy = Math.floor(yTop / 88) * 88; gy < yBot; gy += 88) ctx.fillRect(WALL, gy, W - 2 * WALL, 2);
0842    ctx.strokeStyle = 'rgba(140,200,255,0.05)'; ctx.lineWidth = 10; ctx.lineCap = 'round';
0843    for (let gy = Math.floor(yTop / 300) * 300 + 140; gy < yBot; gy += 300) {
0844      ctx.beginPath();
0845      ctx.moveTo(W / 2 - 46, gy); ctx.lineTo(W / 2, gy + 34); ctx.lineTo(W / 2 + 46, gy);
0846      ctx.stroke();
0847    }
0848    ctx.fillStyle = '#16223f'; ctx.fillRect(0, yTop, WALL, yBot - yTop); ctx.fillRect(W - WALL, yTop, WALL, yB
     ↪ ot - yTop);
0849    ctx.fillStyle = 'rgba(80,220,255,0.4)';
0850    ctx.fillRect(WALL - 3, yTop, 3, yBot - yTop); ctx.fillRect(W - WALL, yTop, 3, yBot - yTop);
0851    drawBand(170, '起点', 'rgba(230,240,255,0.75)', yTop, yBot);
0852    if (trackLen < yBot) drawFinish();
0853    const [dw0, dw1] = obsWindow(yTop - 200, yBot + 200);
0854    for (let k = dw0; k < dw1; k++) {
0855      const o = track.obstacles[k];
0856      if (o._cy < yTop - 180 || o._cy > yBot + 180) continue;
0857      drawObstacle(o, yTop, yBot);
0858    }
0859    for (const b of balls) {
0860      if (b.trail.length < 6 || b.falling !== null) continue;
0861      if (b.y < yTop - 80 || b.y > yBot + 80) continue;
0862      ctx.strokeStyle = b.color; ctx.lineCap = 'round';
0863      const n = b.trail.length / 2;
0864      for (let band = 0; band < 3; band++) {
0865        const i0 = Math.max(2, Math.floor(n * band / 3) * 2);
0866        const i1 = Math.min(b.trail.length - 2, Math.floor(n * (band + 1) / 3) * 2);
0867        if (i1 <= i0) continue;
0868        ctx.globalAlpha = ((band + 1) / 3) * 0.3;
0869        ctx.lineWidth = ((band + 1) / 3) * 7;
0870        ctx.beginPath();
0871        ctx.moveTo(b.trail[i0 - 2], b.trail[i0 - 1]);
0872        for (let i = i0; i <= i1; i += 2) ctx.lineTo(b.trail[i], b.trail[i + 1]);
0873        ctx.stroke();
0874      }
0875      ctx.globalAlpha = 1;
0876    }
0877    for (const b of balls) { if (b.y < yTop - 60 || b.y > yBot + 60) continue; drawBall(b); }
0878    for (const p of particles) {
```

```
    ctx.globalAlpha = Math.max(0, p.life / p.ttl);
    ctx.fillStyle = p.color;
    ctx.beginPath(); ctx.arc(p.x, p.y, p.r, 0, TAU); ctx.fill();
  }
  ctx.globalAlpha = 1;
}
function drawBand(y, label, color, yTop, yBot) {
  if (y + 120 < yTop || y > yBot) return;
  for (let i = 0; i < Math.ceil((W - 2 * WALL) / 22); i++)
    for (let j = 0; j < 2; j++) {
      ctx.fillStyle = (i + j) % 2 ? color : '#12182b';
      ctx.fillRect(WALL + i * 22, y + j * 11, Math.min(22, W - WALL - (WALL + i * 22)), 11);
    }
  ctx.fillStyle = 'rgba(160,200,255,0.55)';
  ctx.font = 'bold 26px sans-serif'; ctx.textAlign = 'center';
  ctx.fillText(label, W / 2, y + 46);
}
function drawFinish() {
  const y = trackLen;
  for (let i = 0; i < Math.ceil((W - 2 * WALL) / 24); i++)
    for (let j = 0; j < 2; j++) {
      ctx.fillStyle = (i + j) % 2 ? '#e6f0ff' : '#111a30';
      ctx.fillRect(WALL + i * 24, y + j * 24, Math.min(24, W - WALL - (WALL + i * 24)), 24);
    }
  ctx.save();
  ctx.shadowColor = '#6cd0ff'; ctx.shadowBlur = 16;
  ctx.fillStyle = '#8fe0ff'; ctx.font = 'bold 40px sans-serif'; ctx.textAlign = 'center';
  ctx.fillText('终 点', W / 2, y + 72);
  ctx.restore();
}
function drawObstacle(o, yTop, yBot) {
  const oy = o._cy;
  if (oy < yTop - 180 || oy > yBot + 180) return;
  if (o.type === 'rot' || o.type === 'piston') ensureKin(o);
  if (o.type === 'rect') {
    fillRoundRect(o.x, o.y, o.w, o.h, 6, '#26375f', 'rgba(140,190,255,0.5)', 2);
  } else if (o.type === 'piston') {
    const r = o._rect; if (!r) return;
    fillRoundRect(r.x, r.y, r.w, r.h, 5, '#2c3f6e', 'rgba(140,190,255,0.4)', 2);
    ctx.fillStyle = '#ffd54a';
    if (o.side === 'L') ctx.fillRect(r.x + r.w - 4, r.y, 4, r.h); else ctx.fillRect(r.x, r.y, 4, r.h);
    ctx.strokeStyle = 'rgba(255,213,74,0.5)'; ctx.lineWidth = 3; ctx.lineCap = 'round';
    const fx = o.side === 'L' ? r.x + r.w - 26 : r.x + 26, dir = o.side === 'L' ? -1 : 1;
    for (let k = -1; k <= 1; k++) {
      ctx.beginPath();
      ctx.moveTo(fx - dir * 8, r.y + r.h / 2 + k * 10 - 5); ctx.lineTo(fx + dir * 4, r.y + r.h / 2 + k * 10)
↪ ;
      ctx.lineTo(fx - dir * 8, r.y + r.h / 2 + k * 10 + 5); ctx.stroke();
    }
  } else if (o.type === 'rot') {
```

```
    ctx.save(); ctx.translate(o.cx, o.cy); ctx.rotate(o._a);
    const active = o.mode !== 'static';
    fillRoundRect(-o.len / 2, -o.thick / 2, o.len, o.thick, o.thick / 2,
      active ? '#ff6b9d' : '#3b4f7d', active ? 'rgba(255,209,230,0.9)' : 'rgba(160,190,240,0.5)', active ? 3
  ↪ : 2);
    ctx.restore();
    ctx.fillStyle = '#ffd54a'; ctx.strokeStyle = '#7a5a00'; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.arc(o.cx, o.cy, 7, 0, TAU); ctx.fill(); ctx.stroke();
  } else if (o.type === 'peg') {
    ctx.fillStyle = '#a9c4f5'; ctx.strokeStyle = 'rgba(235,245,255,0.6)'; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r, 0, TAU); ctx.fill(); ctx.stroke();
    ctx.fillStyle = 'rgba(255,255,255,0.65)';
    ctx.beginPath(); ctx.arc(o.cx - o.r * 0.3, o.cy - o.r * 0.3, o.r * 0.22, 0, TAU); ctx.fill();
  } else if (o.type === 'bumper') {
    const hit = simTime - (o.hitT || -9) < 0.25;
    if (hit) { ctx.shadowColor = '#ff5d5d'; ctx.shadowBlur = 22; }
    ctx.fillStyle = '#3a1620'; ctx.strokeStyle = '#ff5d5d'; ctx.lineWidth = 4;
    ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r, 0, TAU); ctx.fill(); ctx.stroke();
    ctx.shadowBlur = 0;
    ctx.fillStyle = hit ? '#ff8d8d' : '#ff5d5d';
    ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r * 0.45, 0, TAU); ctx.fill();
  } else if (o.type === 'hole') {
    const g = ctx.createRadialGradient(o.cx, o.cy, 2, o.cx, o.cy, o.r);
    g.addColorStop(0, '#000'); g.addColorStop(0.75, '#05070f'); g.addColorStop(1, '#101a33');
    ctx.fillStyle = g;
    ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r, 0, TAU); ctx.fill();
    ctx.strokeStyle = 'rgba(120,160,255,0.25)'; ctx.lineWidth = 2;
    for (let k = 0; k < 2; k++) {
      const a0 = simTime * 1.4 + k * Math.PI + o.cx * 0.01;
      ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r * (0.4 + k * 0.28), a0, a0 + 2.1); ctx.stroke();
    }
    ctx.strokeStyle = 'rgba(140,190,255,0.4)'; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.arc(o.cx, o.cy, o.r, 0, TAU); ctx.stroke();
  }
}
const ballGradCache = {};
function ballGrad(color) {
  let g = ballGradCache[color];
  if (!g) {
    g = ctx.createRadialGradient(-R * 0.35, -R * 0.4, R * 0.15, 0, 0, R);
    g.addColorStop(0, '#ffffff'); g.addColorStop(0.25, color); g.addColorStop(1, shade(color, 0.45));
    ballGradCache[color] = g;
  }
  return g;
}
const nameCache = {};
function drawNameTag(b, hl) {
  const key = b.name + (hl ? '#h' : '');
  let c = nameCache[key];
  if (!c) {
```

```
    const w = Math.ceil(b.name.length * 13 + 10), h = 22;
    c = wx.createCanvas();
    c.width = w * 2; c.height = h * 2; c._w = w; c._h = h;
    const g = c.getContext('2d'); g.scale(2, 2);
    g.font = (hl ? 'bold ' : '') + '12px sans-serif';
    g.textAlign = 'center'; g.textBaseline = 'middle';
    g.lineWidth = 3; g.strokeStyle = 'rgba(5,8,15,0.85)';
    g.strokeText(b.name, w / 2, h / 2 + 1);
    g.fillStyle = hl ? '#ffe9a8' : 'rgba(210,225,250,0.85)';
    g.fillText(b.name, w / 2, h / 2 + 1);
    nameCache[key] = c;
  }
  ctx.drawImage(c, b.x - c._w / 2, b.y - R - 10 - c._h + 5, c._w, c._h);
}
function drawBall(b) {
  const s = b.fallScale;
  if (s <= 0.06) return;
  const followed = mode === 'spectate' && players.includes(b);
  ctx.save();
  ctx.translate(b.x, b.y); ctx.scale(s, s);
  ctx.fillStyle = 'rgba(0,0,0,0.35)';
  ctx.save();
  ctx.translate(4, 7); ctx.scale(1.02, 0.92);
  ctx.beginPath(); ctx.arc(0, 0, R, 0, TAU); ctx.fill();
  ctx.restore();
  if (b.isPlayer || followed) { ctx.shadowColor = b.color; ctx.shadowBlur = 20; }
  ctx.fillStyle = ballGrad(b.color);
  ctx.beginPath(); ctx.arc(0, 0, R, 0, TAU); ctx.fill();
  ctx.shadowBlur = 0;
  ctx.strokeStyle = shade(b.color, 0.3); ctx.lineWidth = 1.5;
  ctx.beginPath(); ctx.arc(0, 0, R - 0.7, 0, TAU); ctx.stroke();
  ctx.fillStyle = 'rgba(255,255,255,0.4)';
  for (const off of [0, Math.PI]) {
    ctx.beginPath();
    ctx.arc(Math.cos(b.spin + off) * R * 0.5, Math.sin(b.spin + off) * R * 0.5, 2.5, 0, TAU); ctx.fill();
  }
  if (Math.hypot(b.vx, b.vy) > 50) {
    ctx.save();
    ctx.rotate(Math.atan2(b.vy, b.vx));
    ctx.fillStyle = 'rgba(255,255,255,0.62)';
    ctx.beginPath();
    ctx.moveTo(R * 0.66, 0); ctx.lineTo(R * 0.2, -R * 0.4); ctx.lineTo(R * 0.33, 0); ctx.lineTo(R * 0.2, R *
↪ 0.4);
    ctx.closePath(); ctx.fill();
    ctx.restore();
  }
  ctx.fillStyle = 'rgba(255,255,255,0.85)';
  ctx.beginPath(); ctx.arc(-R * 0.35, -R * 0.4, 3.4, 0, TAU); ctx.fill();
  ctx.restore();
  drawNameTag(b, b.isPlayer || followed);
```

```
    if ((b.isPlayer || followed) && state === 'race') {
      const bob = Math.sin(simTime * 5) * 3;
      ctx.fillStyle = b.color;
      ctx.beginPath();
      ctx.moveTo(b.x, b.y - R - 30 + bob); ctx.lineTo(b.x - 7, b.y - R - 40 + bob); ctx.lineTo(b.x + 7, b.y - R - 40 + bob);
      ctx.closePath(); ctx.fill();
    }
  }
  function shade(hex, f) {
    const n = parseInt(hex.slice(1), 16);
    const r = (n >> 16) & 255, g = (n >> 8) & 255, b = n & 255;
    return `rgb(${r * f | 0},${g * f | 0},${b * f | 0})`;
  }

  /* ================ 单个视角 ================ */
  function renderView(vx, vy, vw, vh, p, sIdx) {
    const sc = Math.min(vw / (W + 70), vh / 760);
    const viewW = vw / sc, viewH = vh / sc;
    const camX = W / 2 - viewW / 2;
    let cy = 0, shk = 0;
    if (!p) { cy = (menuCam % (trackLen + 600)) - 100; }
    else {
      const target = p.y + p.vy * 0.22 - viewH * 0.38;
      if (p._camY === null) p._camY = target - 60;
      p._camY += (target - p._camY) * 0.12;
      cy = clamp(p._camY, -80, trackLen + 700 - viewH);
      shk = p._shake; p._shake *= 0.88;
    }
    ctx.save();
    ctx.beginPath(); ctx.rect(vx, vy, vw, vh); ctx.clip();
    ctx.translate(vx, vy);
    ctx.save();
    ctx.translate((Math.random() - 0.5) * shk, (Math.random() - 0.5) * shk);
    ctx.scale(sc, sc); ctx.translate(-camX, -cy);
    drawWorld(cy - 100, cy + viewH + 100);
    ctx.restore();
    if (p) drawViewHUD(p, vx, vy, vw, vh, sIdx);
    if (state === 'countdown') drawCountdown(vx + vw / 2, vy + vh / 2);
    else if (state === 'race' && raceTime - goT < 0.9) drawGo(vx + vw / 2, vy + vh / 2);
    ctx.restore();
  }
  function drawViewHUD(p, vx, vy, vw, vh, sIdx) {
    sIdx = sIdx || 0;
    const st = standings();
    const rank = st.indexOf(p) + 1;
    if (mode === '2p' || (mode === 'spectate' && spectateSplit)) {
      const isSp = mode === 'spectate';
      const active = !isSp || sIdx === 1;
      ctx.fillStyle = 'rgba(8,14,28,0.66)';
```

```
    pathRoundRect(vx + 10, vy + 44, isSp ? 236 : 178, 56, 10); ctx.fill();
    ctx.strokeStyle = isSp && active ? 'rgba(255,213,74,0.85)' : 'rgba(90,130,200,0.35)';
    ctx.lineWidth = isSp && active ? 2 : 1; ctx.stroke(); ctx.lineWidth = 2;
    ctx.fillStyle = p.color;
    ctx.beginPath(); ctx.arc(vx + 30, vy + 63, 6, 0, TAU); ctx.fill();
    ctx.fillStyle = active ? '#e8f0ff' : '#93a7c9';
    ctx.font = 'bold 16px sans-serif'; ctx.textAlign = 'left';
    const label = isSp ? (sIdx === 0 ? `屏1 · 榜首 ${p.name}` : `屏2 · ${p.name}`) : p.name;
    ctx.fillText(`${label}  第 ${rank} 名${isSp && active ? ' ▸点击' : ''}`, vx + 44, vy + 69);
    ctx.fillStyle = '#9fb6df'; ctx.font = '12px sans-serif';
    ctx.fillText(`${fmtT(raceTime)} · ${(Math.hypot(p.vx, p.vy) / 10).toFixed(1)} m/s`, vx + 26, vy + 90);
    return;
  }
  const spect = mode === 'spectate';
  ctx.fillStyle = 'rgba(8,14,28,0.62)';
  pathRoundRect(14, 46, spect ? 226 : 196, spect ? 124 : 108, 12); ctx.fill();
  ctx.strokeStyle = 'rgba(90,130,200,0.35)'; ctx.stroke();
  ctx.textAlign = 'left';
  ctx.fillStyle = p.color;
  ctx.beginPath(); ctx.arc(spect ? 36 : 30, 70, 7, 0, TAU); ctx.fill();
  ctx.fillStyle = spect ? '#e8f0ff' : (rank === 1 ? '#ffd54a' : '#e8f0ff');
  ctx.font = 'bold 26px sans-serif';
  ctx.fillText(spect ? `观赛 · ${p.name}` : `第 ${rank} 名`, spect ? 52 : 30, spect ? 79 : 86);
  ctx.font = '14px sans-serif'; ctx.fillStyle = '#9fb6df';
  if (spect) {
    ctx.fillText(`名次  第 ${rank} 名`, 30, 108);
    ctx.fillText(`时间  ${fmtT(raceTime)}  速度  ${(Math.hypot(p.vx, p.vy) / 10).toFixed(1)} m/s`, 30, 132);
    ctx.fillStyle = '#66809f'; ctx.font = '12px sans-serif';
    ctx.fillText('点击排名或点按画面切换视角', 30, 158);
  } else {
    ctx.fillText(`时间  ${fmtT(raceTime)}`, 30, 112);
    ctx.fillText(`速度  ${(Math.hypot(p.vx, p.vy) / 10).toFixed(1)} m/s`, 30, 138);
  }
  const bx = vw - 186;
  ctx.fillStyle = 'rgba(8,14,28,0.62)';
  pathRoundRect(bx - 14, 46, 200, 25 + balls.length * 26, 12); ctx.fill();
  ctx.strokeStyle = 'rgba(90,130,200,0.35)'; ctx.stroke();
  ctx.font = 'bold 13px sans-serif'; ctx.fillStyle = '#7e97c4';
  ctx.fillText('实时排名', bx, 70);
  st.forEach((b, i) => {
    const y = 94 + i * 26;
    const me = spect ? b === players[0] : b.isPlayer;
    if (me) { ctx.fillStyle = 'rgba(255,213,74,0.12)'; ctx.fillRect(bx - 8, y - 15, 184, 22); }
    if (spect) clickTargets.push({ x: bx - 8, y: y - 15, w: 184, h: 22, act: 'follow', i: balls.indexOf(b) }
↪ );
    ctx.fillStyle = b.color;
    ctx.beginPath(); ctx.arc(bx, y - 5, 6, 0, TAU); ctx.fill();
    ctx.fillStyle = me ? '#ffe9a8' : '#c3d3ef';
    ctx.font = (me ? 'bold ' : '') + '13px sans-serif';
    ctx.fillText(`${i + 1}  ${b.name}`, bx + 14, y);
```

```
      ctx.textAlign = 'right';
      ctx.fillStyle = '#7e97c4'; ctx.font = '12px sans-serif';
      ctx.fillText(b.finished ? fmtT(b.finishTime) : `剩${Math.max(0, (trackLen - b.y) / 10) | 0}m`, bx + 168, y);
      ctx.textAlign = 'left';
    });
  }
  function drawCountdown(cx, cy) {
    const n = Math.ceil(countdownT);
    const frac = countdownT - Math.floor(countdownT);
    ctx.fillStyle = 'rgba(4,8,18,0.35)'; ctx.fillRect(0, 0, cw, ch);
    ctx.save();
    ctx.translate(cx, cy);
    const sc = 0.8 + 0.5 * (1 - frac);
    ctx.scale(sc, sc);
    ctx.shadowColor = '#6cd0ff'; ctx.shadowBlur = 30;
    ctx.fillStyle = '#eaf4ff';
    ctx.font = 'bold 110px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText(n > 0 ? n : '', 0, 0);
    ctx.restore();
    ctx.textBaseline = 'alphabetic';
  }
  function drawGo(cx, cy) {
    const t = (raceTime - goT) / 0.9;
    ctx.save();
    ctx.translate(cx, cy); ctx.scale(0.8 + t * 0.5, 0.8 + t * 0.5);
    ctx.globalAlpha = 1 - t;
    ctx.shadowColor = '#ffd54a'; ctx.shadowBlur = 30;
    ctx.fillStyle = '#ffd54a'; ctx.font = 'bold 100px sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText('GO!', 0, 0);
    ctx.restore();
    ctx.textBaseline = 'alphabetic'; ctx.globalAlpha = 1;
  }
  function drawMinimap() {
    const bw = Math.min(cw * 0.62, 860), bh = 14;
    const bx = (cw - bw) / 2, by = 12;
    ctx.fillStyle = 'rgba(8,14,28,0.72)';
    pathRoundRect(bx - 10, by - 7, bw + 20, bh + 14, 12); ctx.fill();
    ctx.strokeStyle = 'rgba(90,130,200,0.4)'; ctx.lineWidth = 1; ctx.stroke();
    const x0 = bx + 6, x1 = bx + bw - 12, my = by + bh / 2;
    ctx.strokeStyle = 'rgba(120,160,220,0.4)'; ctx.lineWidth = 3; ctx.lineCap = 'round';
    ctx.beginPath(); ctx.moveTo(x0, my); ctx.lineTo(x1, my); ctx.stroke();
    ctx.fillStyle = 'rgba(200,220,255,0.6)';
    ctx.beginPath(); ctx.arc(x0, my, 3, 0, TAU); ctx.fill();
    ctx.strokeStyle = '#e8f0ff'; ctx.lineWidth = 1.5;
    ctx.beginPath(); ctx.moveTo(x1 + 7, my - 8); ctx.lineTo(x1 + 7, my + 8); ctx.stroke();
    ctx.fillStyle = '#ff5d5d';
    ctx.beginPath(); ctx.moveTo(x1 + 7, my - 8); ctx.lineTo(x1 + 16, my - 5); ctx.lineTo(x1 + 7, my - 2); ctx.closePath(); ctx.fill();
```

```
  for (const b of balls) {
    const t = clamp(b.y / trackLen, 0, 1);
    const x = lerp(x0, x1, t);
    const isCam = players.includes(b);
    if (isCam) { ctx.fillStyle = '#ffffff'; ctx.beginPath(); ctx.arc(x, my, 7.5, 0, TAU); ctx.fill(); }
    ctx.fillStyle = b.color;
    ctx.beginPath(); ctx.arc(x, my, 5, 0, TAU); ctx.fill();
  }
}
function drawBrand() {
  ctx.fillStyle = 'rgba(142,166,207,0.72)';
  ctx.font = '12px sans-serif'; ctx.textAlign = 'left'; ctx.textBaseline = 'alphabetic';
  ctx.fillText('赛球弹珠竞速', 14, ch - 14);
}
function drawJoysticks() {
  for (const s of joySlots()) {
    const j = joys[s];
    const base = joyBase(s);
    const bx = base[0], by = base[1];
    ctx.globalAlpha = j.pid !== null ? 0.9 : 0.45;
    ctx.beginPath(); ctx.arc(bx, by, 66, 0, TAU);
    ctx.fillStyle = 'rgba(14,23,44,0.6)'; ctx.fill();
    ctx.strokeStyle = 'rgba(120,170,240,0.75)'; ctx.lineWidth = 2; ctx.stroke();
    ctx.beginPath(); ctx.arc(bx + j.dx, by + j.dy, 30, 0, TAU);
    ctx.fillStyle = s === 0 ? 'rgba(255,213,74,0.85)' : 'rgba(108,208,255,0.85)';
    ctx.fill();
    ctx.globalAlpha = 1;
  }
}
function drawButtons() {
  for (const bt of btnLayout()) {
    const on = btnState[bt.p][bt.key] > 0;
    ctx.globalAlpha = on ? 0.92 : 0.5;
    ctx.beginPath(); ctx.arc(bt.x, bt.y, bt.r, 0, TAU);
    ctx.fillStyle = 'rgba(14,23,44,0.62)'; ctx.fill();
    if (on) { ctx.fillStyle = bt.p === 0 ? 'rgba(255,213,74,0.20)' : 'rgba(108,208,255,0.20)'; ctx.fill(); }
    ctx.strokeStyle = bt.p === 0 ? 'rgba(255,213,74,0.8)' : 'rgba(108,208,255,0.8)';
    ctx.lineWidth = 2; ctx.stroke();
    ctx.fillStyle = on ? '#ffe9a8' : (bt.p === 0 ? 'rgba(255,213,74,0.9)' : 'rgba(108,208,255,0.9)');
    ctx.font = `bold ${Math.round(bt.r * 0.8)}px sans-serif`;
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText(bt.icon, bt.x, bt.y + 2);
    ctx.globalAlpha = 1;
  }
  ctx.textBaseline = 'alphabetic';
}
function drawBackButton() {
  const bw = 152, bh = 32, bx = cw - bw - 14, by = ch - bh - 14;
  ctx.fillStyle = 'rgba(8,14,28,0.66)';
  pathRoundRect(bx, by, bw, bh, 16); ctx.fill();
```

```
  ctx.strokeStyle = 'rgba(90,130,200,0.4)'; ctx.lineWidth = 1; ctx.stroke();
  ctx.fillStyle = '#9fb6df'; ctx.font = '13px sans-serif'; ctx.textAlign = 'center';
  ctx.fillText('返回菜单', bx + bw / 2, by + 21);
  uiHits.push({ x: bx, y: by, w: bw, h: bh, act: 'back' });
}
function drawSettingsButton() {
  const bw = 86, bh = 30, bx = cw / 2 - bw / 2, by = ch - bh - 12;
  ctx.fillStyle = 'rgba(8,14,28,0.66)';
  pathRoundRect(bx, by, bw, bh, 15); ctx.fill();
  ctx.strokeStyle = 'rgba(90,130,200,0.4)'; ctx.lineWidth = 1; ctx.stroke();
  ctx.fillStyle = '#9fb6df'; ctx.font = '13px sans-serif'; ctx.textAlign = 'center';
  ctx.fillText('设置', bx + bw / 2, by + 20);
  uiHits.push({ x: bx, y: by, w: bw, h: bh, act: 'open-settings' });
}
function drawBackPrompt() {
  ctx.fillStyle = 'rgba(4,8,18,0.6)'; ctx.fillRect(0, 0, cw, ch);
  const pw = 380, ph = 160, px = (cw - pw) / 2, py = (ch - ph) / 2;
  fillRoundRect(px, py, pw, ph, 14, '#0e1730', '#2b3f6b', 1);
  ctx.textAlign = 'center';
  ctx.fillStyle = '#e8f0ff'; ctx.font = 'bold 20px sans-serif';
  ctx.fillText('返回主菜单? ', cw / 2, py + 46);
  ctx.fillStyle = '#8ea6cf'; ctx.font = '14px sans-serif';
  ctx.fillText('当前比赛进度将不会保存', cw / 2, py + 74);
  const bw = 130, bh = 38, gy = py + 94;
  const nx = cw / 2 - bw - 12, yx = cw / 2 + 12;
  fillRoundRect(nx, gy, bw, bh, 10, '#0d1730', '#33508a', 1);
  ctx.fillStyle = '#9fb6df'; ctx.font = 'bold 15px sans-serif';
  ctx.fillText('取消', nx + bw / 2, gy + 24);
  uiHits.push({ x: nx, y: gy, w: bw, h: bh, act: 'prompt-no' });
  fillRoundRect(yx, gy, bw, bh, 10, 'rgba(255,213,74,0.15)', '#ffd54a', 1);
  ctx.fillStyle = '#ffd54a';
  ctx.fillText('确定返回', yx + bw / 2, gy + 24);
  uiHits.push({ x: yx, y: gy, w: bw, h: bh, act: 'prompt-yes' });
}

/* ================= Canvas UI：菜单 / 结算 / 设置 ================= */
let uiHits = [];
let uiScale = 1, uiOX = 0, uiOY = 0;
const UI_W = 430;
function pushHit(x, y, w, h, act, data) {
  uiHits.push({ x: uiOX + x * uiScale, y: uiOY + y * uiScale, w: w * uiScale, h: h * uiScale, act, data });
}
function beginUILayer(vh) {
  uiScale = Math.min(cw / UI_W, (ch - 16) / vh, 1.2);
  uiOX = (cw - UI_W * uiScale) / 2;
  uiOY = (ch - vh * uiScale) / 2;
  ctx.save();
  ctx.translate(uiOX, uiOY);
  ctx.scale(uiScale, uiScale);
}
```

```
function endUILayer() { ctx.restore(); }
function uiButton(x, y, w, h, label, active, act, data, opts) {
  opts = opts || {};
  const fill = active ? 'rgba(28,58,110,0.95)' : 'rgba(13,23,48,0.95)';
  fillRoundRect(x, y, w, h, opts.radius || h / 2, fill, active ? '#6cd0ff' : '#33508a', active ? 2 : 1);
  ctx.font = (opts.bold ? 'bold ' : '') + (opts.font || 14) + 'px sans-serif';
  ctx.fillStyle = active ? '#ffffff' : '#9fb6df';
  ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText(label, x + w / 2, y + h / 2 + 1);
  ctx.textBaseline = 'alphabetic';
  pushHit(x, y, w, h, act, data);
}
function drawMenuUI() {
  ctx.fillStyle = 'rgba(4,8,18,0.74)'; ctx.fillRect(0, 0, cw, ch);
  beginUILayer(780);
  const px = 14, py = 10, pw = 402, ph = 760;
  fillRoundRect(px, py, pw, ph, 18, 'rgba(10,17,34,0.92)', '#2b3f6b', 1);
  const cx = UI_W / 2;

  // 标题
  const tg = ctx.createLinearGradient(cx - 120, 0, cx + 120, 0);
  tg.addColorStop(0, '#ffd54a'); tg.addColorStop(0.45, '#ff8a5c'); tg.addColorStop(1, '#6cd0ff');
  ctx.font = 'bold 32px sans-serif'; ctx.fillStyle = tg;
  ctx.textAlign = 'center'; ctx.textBaseline = 'alphabetic';
  ctx.fillText('赛球弹珠竞速', cx, 92);
  textCentered('MARBLE RUSH · 弹珠障碍追逐赛', cx, 118, '11px sans-serif', '#8ea6cf');

  // 简介
  wrapText('上帝视角弹珠竞速：冲下超长滑坡赛道，闪避风车、活塞、漏斗与黑洞，撞开对手抢线；赛道长度与 AI 难度自由组合。',
    px + 18, 146, pw - 36, 18, '12px sans-serif', '#c3d3ef');

  const bx0 = px + 16, bwAll = pw - 32;
  // 模式
  textCentered('模 式', cx, 212, '11px sans-serif', '#7e97c4');
  const modes = [['1p', '单人'], ['2p', '双人 · 分屏'], ['spectate', '全AI观赛']];
  const mw = (bwAll - 12) / 3;
  modes.forEach((m, i) => uiButton(bx0 + i * (mw + 6), 222, mw, 34, m[1], mode === m[0], 'mode', m[0]));

  // 赛道长度
  textCentered('赛道长度', cx, 282, '11px sans-serif', '#7e97c4');
  const lens = [['short', '短 ≈40秒'], ['medium', '中 ≈1分30'], ['long', '长 ≈2分钟'], ['xl', '超长 ≈3分钟']];
  const lw = (bwAll - 18) / 4;
  lens.forEach((l, i) => uiButton(bx0 + i * (lw + 6), 292, lw, 32, l[1], lenKey === l[0], 'len', l[0], { font: 12 }));

  // AI 难度
  textCentered('AI 难度（人机强度 · 机关密度）', cx, 346, '11px sans-serif', '#7e97c4');
  const ais = [['easy', '简单'], ['normal', '普通'], ['hard', '困难'], ['nightmare', '噩梦']];
  const aw = (bwAll - 18) / 4;
  ais.forEach((a, i) => uiButton(bx0 + i * (aw + 6), 356, aw, 32, a[1], diffKey === a[0], 'ai', a[0], { font
```

```
    ↪ : 12 }));

  // 小球数量
  textCentered('小球数量 (1-10，颜色各不相同) ', cx, 410, '11px sans-serif', '#7e97c4');
  const step = bwAll / 10;
  for (let i = 1; i <= 10; i++) {
    const ccx = bx0 + step * (i - 0.5), ccy = 434;
    const on = ballCount === i;
    ctx.beginPath(); ctx.arc(ccx, ccy, 15, 0, TAU);
    ctx.fillStyle = on ? 'rgba(28,58,110,0.95)' : 'rgba(13,23,48,0.95)';
    ctx.fill();
    ctx.strokeStyle = on ? '#6cd0ff' : '#33508a'; ctx.lineWidth = on ? 2 : 1; ctx.stroke();
    ctx.font = '13px sans-serif'; ctx.fillStyle = on ? '#fff' : '#9fb6df';
    ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText(String(i), ccx, ccy + 1);
    ctx.textBaseline = 'alphabetic';
    pushHit(ccx - 17, ccy - 17, 34, 34, 'count', i);
  }

  // 选手命名
  textCentered('选手命名（点击方框改名）', cx, 470, '11px sans-serif', '#7e97c4');
  const cols = 5, cellW = (bwAll - 4 * 6) / cols;
  for (let i = 0; i < 10; i++) {
    const r = Math.floor(i / cols), c = i % cols;
    const nx = bx0 + c * (cellW + 6), ny = 480 + r * 36;
    const on = i < ballCount;
    fillRoundRect(nx, ny, cellW, 30, 8, '#0d1730', '#24365e', 1);
    ctx.globalAlpha = on ? 1 : 0.32;
    ctx.fillStyle = BALL_POOL[i].color;
    ctx.beginPath(); ctx.arc(nx + 11, ny + 15, 5, 0, TAU); ctx.fill();
    ctx.font = '11px sans-serif'; ctx.fillStyle = '#e8f0ff';
    ctx.textAlign = 'left'; ctx.textBaseline = 'middle';
    ctx.fillText(customNames[i], nx + 21, ny + 16);
    ctx.textBaseline = 'alphabetic'; ctx.globalAlpha = 1;
    pushHit(nx, ny, cellW, 30, 'name', i);
  }

  // 开始 / 设置
  const sbw = Math.min(260, bwAll - 40);
  const sbx = cx - sbw / 2, sby = 560;
  const sg = ctx.createLinearGradient(sbx, sby, sbx + sbw, sby + 52);
  sg.addColorStop(0, '#ffd54a'); sg.addColorStop(1, '#ffb14a');
  fillRoundRect(sbx, sby, sbw, 52, 12, sg);
  ctx.font = 'bold 20px sans-serif'; ctx.fillStyle = '#08111f';
  ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
  ctx.fillText('开始比赛 ▶', cx, sby + 27);
  ctx.textBaseline = 'alphabetic';
  pushHit(sbx, sby, sbw, 52, 'start');

  uiButton(cx - 65, 628, 130, 34, '设置', false, 'open-settings', null, { font: 14 });
```

```
1369
1370    textCentered(bestText(), cx, 686, '12px sans-serif', '#7e97c4');
1371    textCentered('手机操作：左下摇杆转向（上推刹车）· 双人模式右摇杆为 P2', cx, 716, '11px sans-serif', '#66809f');
1372    textCentered('比赛中点底部「设置」可切换左右按钮 / 调节音效', cx, 736, '11px sans-serif', '#66809f');
1373    endUILayer();
1374  }
1375  function drawResultsUI() {
1376    ctx.fillStyle = 'rgba(4,8,18,0.78)'; ctx.fillRect(0, 0, cw, ch);
1377    beginUILayer(800);
1378    const px = 18, pw = 394;
1379    const py = 100, ph = 680;
1380    fillRoundRect(px, py, pw, ph, 18, 'rgba(10,17,34,0.96)', '#2b3f6b', 1);
1381    const cx = UI_W / 2;
1382    textCentered('赛球弹珠竞速', cx, py + 32, '12px sans-serif', '#6cd0ff');
1383    ctx.font = 'bold 26px sans-serif'; ctx.fillStyle = '#e8f0ff'; ctx.textAlign = 'center';
1384    ctx.fillText(resTitleText, cx, py + 72);
1385    wrapText(resNoteText, px + 22, py + 98, pw - 44, 18, '12px sans-serif', '#8ea6cf');
1386    const st = standings();
1387    const top = py + 150;
1388    ctx.font = 'bold 12px sans-serif'; ctx.fillStyle = '#7e97c4'; ctx.textAlign = 'left';
1389    ctx.fillText('名次', px + 26, top);
1390    ctx.fillText('选手', px + 86, top);
1391    ctx.textAlign = 'right';
1392    ctx.fillText('用时', px + pw - 90, top);
1393    ctx.fillText('落坑', px + pw - 30, top);
1394    ctx.strokeStyle = '#2b3f6b'; ctx.lineWidth = 1;
1395    ctx.beginPath(); ctx.moveTo(px + 22, top + 8); ctx.lineTo(px + pw - 22, top + 8); ctx.stroke();
1396    const rowH = st.length > 6 ? 27 : 34;
1397    st.forEach((b, i) => {
1398      const y = top + 34 + i * rowH;
1399      const me = b.isPlayer || (mode === 'spectate' && players.includes(b));
1400      if (me) { ctx.fillStyle = 'rgba(255,213,74,0.10)'; ctx.fillRect(px + 16, y - 20, pw - 32, 30); }
1401      ctx.textAlign = 'left';
1402      ctx.font = '13px sans-serif'; ctx.fillStyle = me ? '#ffe9a8' : '#c3d3ef';
1403      const medal = i === 0 ? '🥇' : i === 1 ? '🥈' : i === 2 ? '🥉' : String(i + 1);
1404      ctx.fillText(medal, px + 26, y);
1405      ctx.fillStyle = b.color;
1406      ctx.beginPath(); ctx.arc(px + 92, y - 4, 5, 0, TAU); ctx.fill();
1407      ctx.fillStyle = me ? '#ffe9a8' : '#c3d3ef';
1408      ctx.fillText(b.name, px + 104, y);
1409      ctx.textAlign = 'right';
1410      ctx.fillText(b.finished ? fmtT(b.finishTime) : '未完赛', px + pw - 90, y);
1411      ctx.fillText(String(b.falls), px + pw - 30, y);
1412    });
1413    ctx.textAlign = 'center';
1414    const by = top + 34 + Math.min(st.length, 10) * rowH + 20;
1415    uiButton(px + 40, by, pw - 80, 46, '再来一局', true, 'again', null, { bold: true, font: 17, radius: 12 });
1416    uiButton(px + 40, by + 56, pw - 80, 38, '换新赛道', false, 'new', null, { radius: 10 });
1417    uiButton(px + 40, by + 102, pw - 80, 38, '返回菜单', false, 'menu', null, { radius: 10 });
1418    endUILayer();
```

```
}
function drawSettingsUI() {
  ctx.fillStyle = 'rgba(4,8,18,0.78)'; ctx.fillRect(0, 0, cw, ch);
  beginUILayer(660);
  const px = 20, pw = 390, py = 110, ph = 520;
  fillRoundRect(px, py, pw, ph, 18, 'rgba(10,17,34,0.96)', '#2b3f6b', 1);
  const cx = UI_W / 2;
  textCentered('赛球弹珠竞速', cx, py + 32, '12px sans-serif', '#6cd0ff');
  ctx.font = 'bold 26px sans-serif'; ctx.fillStyle = '#e8f0ff'; ctx.textAlign = 'center';
  ctx.fillText('设 置', cx, py + 72);
  textCentered('操作方式与音效会自动保存在本机', cx, py + 98, '12px sans-serif', '#8ea6cf');

  textCentered('操作方式（手机触屏）', cx, py + 140, '11px sans-serif', '#7e97c4');
  const bwAll = pw - 60, bw = (bwAll - 10) / 2, bx0 = px + 30;
  uiButton(bx0, py + 152, bw, 40, '虚拟摇杆', settings.ctrl === 'joy', 'ctrl', 'joy');
  uiButton(bx0 + bw + 10, py + 152, bw, 40, '左右按钮', settings.ctrl === 'btn', 'ctrl', 'btn');
  const desc = settings.ctrl === 'btn'
    ? '按钮：屏幕下方 ◀=左转（左）、▶=右转（右）；双人模式 P1 在左半边、P2 在右半边'
    : '摇杆：左下角推动转向（推得越满转得越急），上推刹车；双人模式右下角为 P2';
  wrapText(desc, px + 26, py + 222, pw - 52, 17, '11px sans-serif', '#7ec9a9');

  textCentered('音效音量', cx, py + 274, '11px sans-serif', '#7e97c4');
  const sx0 = px + 30, sx1 = px + pw - 78, sy = py + 296;
  ctx.strokeStyle = '#33508a'; ctx.lineWidth = 6; ctx.lineCap = 'round';
  ctx.beginPath(); ctx.moveTo(sx0, sy); ctx.lineTo(sx1, sy); ctx.stroke();
  const kx = sx0 + (sx1 - sx0) * settings.vol;
  ctx.strokeStyle = '#ffd54a'; ctx.lineWidth = 6;
  ctx.beginPath(); ctx.moveTo(sx0, sy); ctx.lineTo(kx, sy); ctx.stroke();
  ctx.fillStyle = '#ffd54a';
  ctx.beginPath(); ctx.arc(kx, sy, 11, 0, TAU); ctx.fill();
  ctx.font = 'bold 14px sans-serif'; ctx.fillStyle = '#ffe9a8'; ctx.textAlign = 'left';
  ctx.fillText(Math.round(settings.vol * 100) + '%', sx1 + 16, sy + 5);
  pushHit(px + 20, sy - 22, pw - 40, 44, 'volume');

  const hbw = (bwAll - 10) / 2;
  uiButton(bx0, py + 334, hbw, 38, settings.mute ? '音效 关' : '音效 开', !settings.mute, 'mute', null);
  uiButton(bx0 + hbw + 10, py + 334, hbw, 38, '🔊 试听', false, 'test', null);
  uiButton(px + 50, py + 400, pw - 100, 44, '关闭 ✕', false, 'close', null, { radius: 12, font: 16 });
  endUILayer();
}

/* ================= 触控 ================= */
function joySlots() {
  if (state === 'menu' || paused || backPrompt || settingsOpen || resultsVisible) return [];
  if (mode === '1p') return [0];
  if (mode === '2p') return [0, 1];
  return [];
}
function joyBase(i) { return i === 0 ? [116, ch - 128] : [cw - 116, ch - 128]; }
function joyHit(x, y) {
```

```
  for (const s of joySlots()) {
    const b = joyBase(s);
    if (Math.hypot(x - b[0], y - b[1]) < 90) return s;
  }
  return -1;
}
function joyMove(j, x, y) {
  const dx = x - j.bx, dy = y - j.by;
  const d = Math.hypot(dx, dy) || 1;
  const cl = Math.min(d, JOY_R);
  j.dx = dx / d * cl; j.dy = dy / d * cl;
  j.x = j.dx / JOY_R; j.y = j.dy / JOY_R;
}
function btnSlots() {
  if (settings.ctrl !== 'btn') return false;
  if (state === 'menu' || paused || backPrompt || settingsOpen || resultsVisible) return false;
  return mode === '1p' || mode === '2p';
}
function btnLayout() {
  const r = cw < 430 ? 46 : 54, cy = ch - (r + 66);
  if (mode !== '2p')
    return [{ p: 0, key: 'left', x: 88, y: cy, r, icon: '◀' },
            { p: 0, key: 'right', x: cw - 88, y: cy, r, icon: '▶' }];
  const g = r * 2.4;
  return [{ p: 0, key: 'left', x: 72, y: cy, r, icon: '◀' },
          { p: 0, key: 'right', x: 72 + g, y: cy, r, icon: '▶' },
          { p: 1, key: 'left', x: cw - 72 - g, y: cy, r, icon: '◀' },
          { p: 1, key: 'right', x: cw - 72, y: cy, r, icon: '▶' }];
}
function btnHit(x, y) {
  if (!btnSlots()) return null;
  for (const bt of btnLayout()) if (Math.hypot(x - bt.x, y - bt.y) < bt.r + 16) return bt;
  return null;
}
function hitTest(x, y) {
  for (let i = uiHits.length - 1; i >= 0; i--) {
    const h = uiHits[i];
    if (x >= h.x && x <= h.x + h.w && y >= h.y && y <= h.y + h.h) return h;
  }
  return null;
}
function editName(i) {
  editingName = i;
  try {
    wx.showKeyboard({ defaultValue: customNames[i] || '', maxLength: 6, multiple: false, confirmHold: false,
↪  confirmType: 'done' });
  } catch (e) { editingName = -1; }
}
function applyName(v) {
  if (editingName < 0) return;
```

```
    const t = (v || '').trim();
    customNames[editingName] = t ? t.slice(0, 6) : BALL_POOL[editingName].name;
    saveRoster();
    editingName = -1;
  }
  if (wx.onKeyboardConfirm) wx.onKeyboardConfirm(e => { applyName(e.value); try { wx.hideKeyboard(); } catch (
  ↪ err) {} });
  if (wx.onKeyboardComplete) wx.onKeyboardComplete(e => { applyName(e.value); });

  let sliderDragId = null;
  function handleUIHit(h, x, y) {
    if (!h) return false;
    switch (h.act) {
      case 'mode': mode = h.data; return true;
      case 'len': lenKey = h.data; return true;
      case 'ai': diffKey = h.data; return true;
      case 'count': ballCount = h.data; saveRoster(); return true;
      case 'name': editName(h.data); return true;
      case 'start': startRace(false); return true;
      case 'open-settings':
        settingsOpen = true;
        for (const j of joys) { j.pid = null; j.dx = j.dy = j.x = j.y = 0; }
        btnPresses.clear(); syncBtnState();
        return true;
      case 'again': startRace(false); return true;
      case 'new': startRace(true); return true;
      case 'menu': goMenu(); return true;
      case 'ctrl': settings.ctrl = h.data; btnPresses.clear(); syncBtnState(); saveSettings(); return true;
      case 'mute': settings.mute = !settings.mute; muted = settings.mute; saveSettings(); if (!settings.mute)
  ↪ { ensureAudio(); sndBeep(); } return true;
      case 'test': ensureAudio(); sndBeep(); return true;
      case 'close': settingsOpen = false; return true;
      case 'volume': updateVolume(x); sliderDragId = -1; return true;
      case 'back': backPrompt = true; return true;
      case 'prompt-no': backPrompt = false; return true;
      case 'prompt-yes': backPrompt = false; goMenu(); return true;
      default: return false;
    }
  }
  function updateVolume(x) {
    const vx = (x - uiOX) / uiScale;
    const sx0 = 20 + 30, sx1 = 20 + 390 - 78;
    settings.vol = clamp((vx - sx0) / (sx1 - sx0), 0, 1);
    applyVolume(); saveSettings();
  }
  function onTouchStart(x, y, id) {
    ensureAudio();
    if (settingsOpen) {
      const h = hitTest(x, y);
      if (h && h.act === 'volume') { sliderDragId = id; updateVolume(x); return; }
```

```
    if (handleUIHit(h, x, y)) return;
    return;
  }
  if (state === 'menu') { handleUIHit(hitTest(x, y), x, y); return; }
  if (resultsVisible) { handleUIHit(hitTest(x, y), x, y); return; }
  if (backPrompt) { handleUIHit(hitTest(x, y), x, y); return; }
  if (settings.ctrl === 'btn') {
    const bt = btnHit(x, y);
    if (bt) { btnPresses.set(id, { p: bt.p, key: bt.key }); syncBtnState(); return; }
  }
  const s = settings.ctrl === 'btn' ? -1 : joyHit(x, y);
  if (s >= 0) {
    const j = joys[s]; const base = joyBase(s);
    j.pid = id; j.bx = base[0]; j.by = base[1];
    joyMove(j, x, y);
    return;
  }
  if (mode === 'spectate') {
    for (const t of clickTargets) {
      if (x < t.x || x > t.x + t.w || y < t.y || y > t.y + t.h) continue;
      if (t.act === 'follow') { players[0] = balls[t.i]; players[0]._camY = null; return; }
    }
  }
  const h = hitTest(x, y);
  if (h) handleUIHit(h, x, y);
}
function onTouchMove(x, y, id) {
  if (sliderDragId === id) { updateVolume(x); return; }
  for (const j of joys) {
    if (j.pid === id) { joyMove(j, x, y); return; }
  }
}
function onTouchEnd(x, y, id) {
  if (sliderDragId === id) sliderDragId = null;
  for (const j of joys) {
    if (j.pid === id) { j.pid = null; j.dx = j.dy = j.x = j.y = 0; }
  }
  if (btnPresses.delete(id)) syncBtnState();
}
wx.onTouchStart(e => { for (const t of e.changedTouches) onTouchStart(t.clientX, t.clientY, t.identifier); }
↪ );
wx.onTouchMove(e => { for (const t of e.touches) onTouchMove(t.clientX, t.clientY, t.identifier); });
const touchEndHandler = e => { for (const t of e.changedTouches) onTouchEnd(t.clientX, t.clientY, t.identifi
↪ er); };
wx.onTouchEnd(touchEndHandler);
wx.onTouchCancel(touchEndHandler);

/* ================= 总渲染 ================= */
function render() {
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
```

```
  clickTargets.length = 0;
  uiHits.length = 0;
  uiScale = 1; uiOX = 0; uiOY = 0;
  const bgGrad = ctx.createLinearGradient(0, 0, 0, ch);
  bgGrad.addColorStop(0, '#070b16'); bgGrad.addColorStop(1, '#04060c');
  ctx.fillStyle = bgGrad; ctx.fillRect(0, 0, cw, ch);
  if (state === 'menu' || resultsVisible) {
    renderView(0, 0, cw, ch, state === 'menu' ? null : players[0], 0);
  } else if (mode === '2p' || (mode === 'spectate' && spectateSplit)) {
    renderView(0, 0, cw / 2 - 2, ch, players[0], 0);
    renderView(cw / 2 + 2, 0, cw / 2 - 2, ch, players[1], 1);
    ctx.fillStyle = '#0a1122'; ctx.fillRect(cw / 2 - 4, 0, 8, ch);
    ctx.fillStyle = 'rgba(108,208,255,0.5)'; ctx.fillRect(cw / 2 - 1, 0, 2, ch);
  } else {
    renderView(0, 0, cw, ch, players[0], 0);
  }
  drawMinimap();
  drawBrand();
  if (state !== 'menu' && !settingsOpen && !resultsVisible && !backPrompt) {
    drawBackButton();
    drawSettingsButton();
  }
  if (state === 'race' && raceTime < 4 && !settingsOpen && !resultsVisible) {
    ctx.textAlign = 'center'; ctx.fillStyle = 'rgba(195,211,239,0.85)';
    ctx.font = '13px sans-serif';
    const tip = mode === '2p' ? 'P1 左 · P2 右 · 撞开对手抢线! '
      : mode === 'spectate' ? '点按排名或画面切换视角'
      : '左右转向 · 上推刹车 · 撞开对手抢线! ';
    ctx.fillText(tip, cw / 2, ch - 58);
  }
  if (touchSeen && settings.ctrl !== 'btn' && !settingsOpen && !resultsVisible) drawJoysticks();
  if (btnSlots()) drawButtons();
  if (backPrompt) drawBackPrompt();
  if (state === 'menu' && !settingsOpen) drawMenuUI();
  if (resultsVisible && !settingsOpen) drawResultsUI();
  if (settingsOpen) drawSettingsUI();
}

/* ================ 主循环 ================ */
let lastTs = 0;
function frame(ts) {
  requestAnimationFrame(frame);
  if (!lastTs) lastTs = ts;
  const dt = clamp((ts - lastTs) / 1000, 0, 0.05); lastTs = ts;
  if (!paused && !backPrompt && !settingsOpen) {
    simTime += dt;
    if (state === 'menu') { menuCam += dt * 80; }
    else if (state === 'countdown') {
      countdownT -= dt;
      const n = Math.ceil(countdownT);
```

```
      if (n < lastCount && n > 0) { sndBeep(); lastCount = n; }
      if (countdownT <= 0) { state = 'race'; goT = raceTime; sndGo(); }
    } else if (state === 'race' || state === 'finishing') {
      raceTime += dt;
      physics(dt / 2); physics(dt / 2);
      if (state === 'finishing') {
        if (mode !== 'spectate' && raceTime - lastResUpdate > 0.25) { renderResults(false); lastResUpdate = raceTime; }
        const done = mode === 'spectate' ? balls.every(x => x.finished) : players.every(p => p.finished);
        if (done || raceTime > finishWaitEnd) renderResults(true);
      }
    }
    for (let i = particles.length - 1; i >= 0; i--) {
      const p = particles[i];
      p.life -= dt; if (p.life <= 0) { particles.splice(i, 1); continue; }
      p.x += p.vx * dt; p.y += p.vy * dt; p.vx *= 0.985; p.vy = p.vy * 0.985 + 120 * dt;
    }
  }
  render();
}

/* ================ 启动 ================ */
loadRoster();
loadSettings();
track = genTrack(seed, AILVLS[diffKey].scale, trackLen);
balls = makeBalls();
requestAnimationFrame(frame);

```